Performance and Functional Testing

Phase 6: Project Testing

1. Testing Purpose

Testing checks that the deployed application behaves as expected and that failures are visible to the user. The test plan covers the main workflow, persistence, external searches, printing and common error paths. No benchmark logs or test run results were supplied, so this document describes the test method and expected outcomes rather than claiming measured performance.

2. Environment and Test Data

Use the deployed frontend and backend URLs or a documented local setup. A representative test case can use Bhopal as the starting city, Kerala as the destination, 21–23 October 2026, a ■20,000 budget, two travellers, interests Nature/Sightseeing/Food and Adventure travel style. Confirm the dates and values before using this case in a recorded demonstration.

3. Functional Test Areas

Deployment smoke test

Open the frontend and confirm that the main page loads. Request the backend health endpoint and record the response and time. A health response is a basic service check, not a substitute for a full itinerary generation test.

Itinerary generation

Submit a valid request and verify that the result includes a summary, day-wise activities, estimated cost and travel tips. Repeat with missing or invalid inputs and confirm that the application gives a clear error rather than displaying malformed output.

Persistence

Save a generated trip, reload the page and open Saved Trips. Select the saved item and confirm that its details appear. Delete a test record and confirm that it no longer appears. Avoid deleting a real trip that is needed for evidence without first taking a screenshot.

External search

Open the hotel search and several restaurant food-preference searches. Check that the browser opens Google Maps with the intended destination and search term. This test confirms the link construction only; it does not validate individual business information.

Print/save

Open the print interface with a generated itinerary and inspect page breaks, missing sections and readability. Save a PDF for evidence if needed.

4. Basic Performance Measurement Method

Record request start and completion times for multiple itinerary generations. Separate backend cold-start delay from model response time where possible. Repeat tests with the same input and record success or failure. Do not report averages or percentiles until enough actual observations have been collected.

5. Error and Reliability Checks

- Try a request when the AI provider is unavailable or quota-limited, if this can be tested safely.
- Check behaviour when Supabase configuration is invalid in a controlled local environment.
- Verify that unknown trip IDs return a suitable error.
- Confirm that invalid restaurant food_type values are rejected.
- Check that the UI remains usable after a failed request.

6. Results Log Template

For each test, record date/time, environment, input, expected outcome, actual outcome, pass/fail and evidence filename. Mark a test as passed only after observing the expected behaviour. The current status should remain “Pending verification” until that evidence is recorded.

7. Conclusion

The plan provides a repeatable test path for the core application. A final submission should include actual results and screenshots or logs, and should avoid unsupported claims about response time, throughput or reliability.